Message Integrity
&
Message Authentication

Objectives

- To define message integrity
- To define message authentication
- To define criteria for a cryptographic hash function
- To introduce general ideas behind cryptographic hash functions
- To discuss the Merkle-Damgard scheme as the basis for iterated hash functions
- To distinguish between two categories of hash functions
- To discuss the structure of SHA-512

11-1 MESSAGE INTEGRITY

The cryptography systems that we have studied so far provide secrecy, or confidentiality, but not integrity. However, there are occasions where we may not even need secrecy but instead must have integrity.

Topics discussed in this section:

11.1 Document and Fingerprint
11.2 Message and Message Digest
11.3 Difference
11.4 Checking Integrity
11.5 Cryptographic Hash Function Criteria

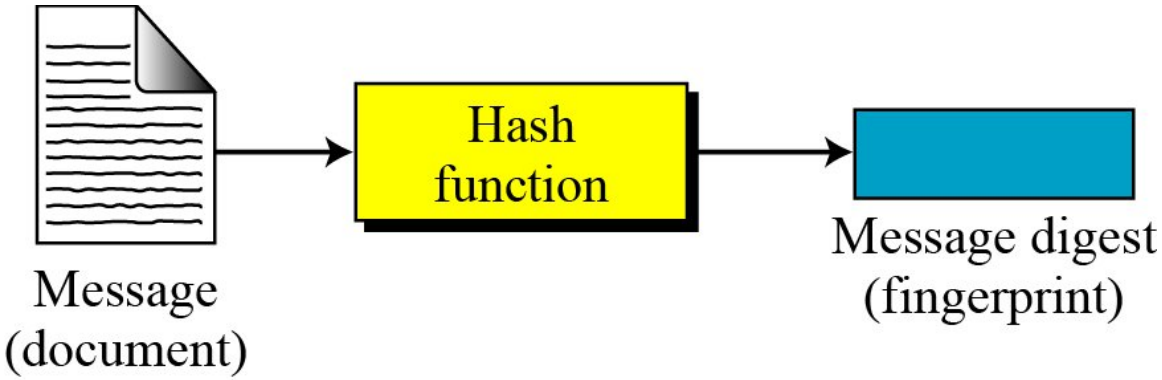
11.1.1 Document and Fingerprint

One way to preserve the integrity of a document is through the use of a fingerprint. If Alice needs to be sure that the contents of her document will not be changed, she can put her fingerprint at the bottom of the document.

11.1.2 Message and Message Digest

The electronic equivalent of the document and fingerprint pair is the message and digest pair.

Figure 11.1 ***Message and digest***



11.1.3 Difference

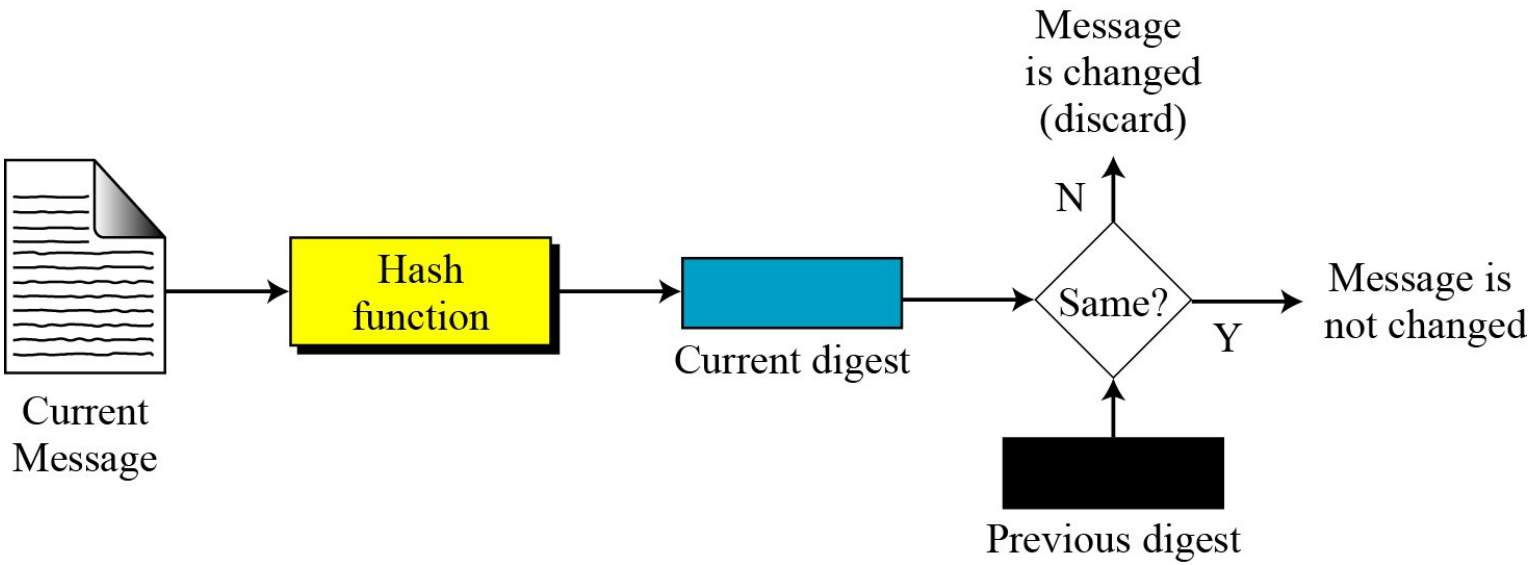
The two pairs (document / fingerprint) and (message / message digest) are similar, with some differences. The document and fingerprint are physically linked together. The message and message digest can be unlinked separately, and, most importantly, the message digest needs to be safe from change.

Note

The message digest needs to be safe from change.

11.1.4 Checking Integrity

Figure 11.2 ***Checking integrity***



Hash Functions

Hash Function Motivation

- Suppose Alice signs M
  - Alice sends M and S = $[M]_{Alice}$ to Bob
  - Bob verifies that M = $\{S\}_{Alice}$
  - Is it OK to just send S?
- If M is big, $[M]_{Alice}$ is costly to compute
- Suppose instead, Alice signs h(M), where h(M) is much smaller than M
  - Alice sends M and S = $[h(M)]_{Alice}$ to Bob
  - Bob verifies that h(M) = $\{S\}_{Alice}$

Crypto Hash Function

- Crypto hash function h(x) must provide
  - **Compression** ❑ output length is small
  - **Efficiency** ❑ h(x) easy to computer for any x
  - **One-way** ❑ given a value y it is infeasible to find an x such that h(x) = y
  - **Weak collision resistance** ❑ given x and h(x), infeasible to find y ≠ x such that h(y) = h(x)
  - **Strong collision resistance** ❑ infeasible to find any x and y, with x ≠ y such that h(x) = h(y)
  - Lots of collisions exist, but hard to find one

Pre-Birthday Problem

- Suppose N people in a room
- How large must N be before the probability someone has same birthday as me is 1/2
  - Solve: $1/2 = 1 - (364/365)^N$ for N
  - Find N = 253

Birthday Problem

- How many people must be in a room before probability is 1/2 that two or more have same birthday?
  - 1 365/365 364/365 (365N+1)/365
  - Set equal to 1/2 and solve: **N = 23**
- Surprising? A paradox?
- Maybe not: “Should be” about sqrt(365) since we compare all **pairs** x and y

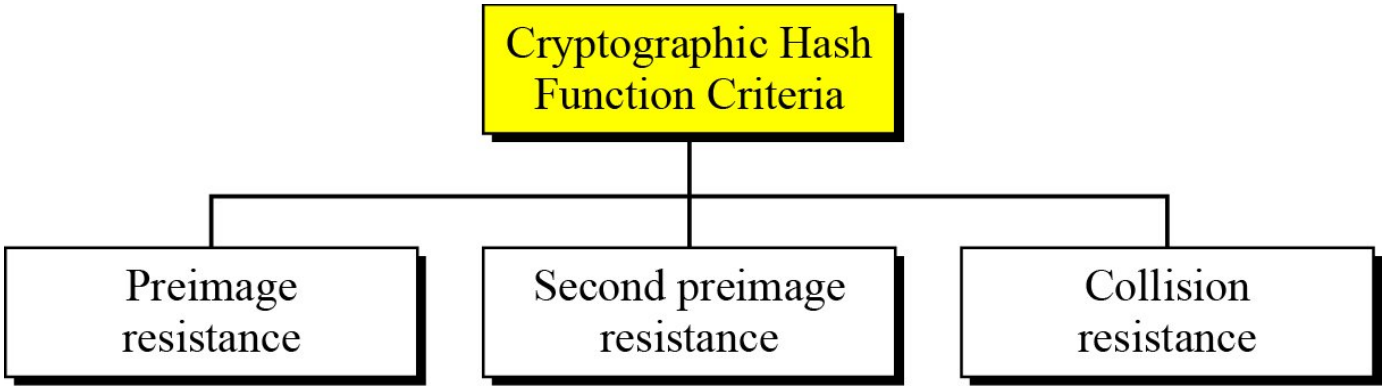
Of Hashes and Birthdays

- If h(x) is N bits, then 2^N different hash values are possible
- $\text{sqrt}(2^N) = 2^{N/2}$
- Therefore, hash about $2^{N/2}$ random values and you expect to find a collision
- **Implication:** secure N bit symmetric key requires $2^{\tilde{N}1}$ work to "break" while secure N bit hash requires $2^{N/2}$ work to "break"

11.1.5 Cryptographic Hash Function Criteria

A cryptographic hash function must satisfy three criteria: preimage resistance, second preimage resistance, and collision resistance.

Figure 11.3 *Criteria of a cryptographic hash function*



11.1.5 Continued

Preimage Resistance

Preimage Attack

Given: y = h(M) **Find: M′ such that y = h(M′)**

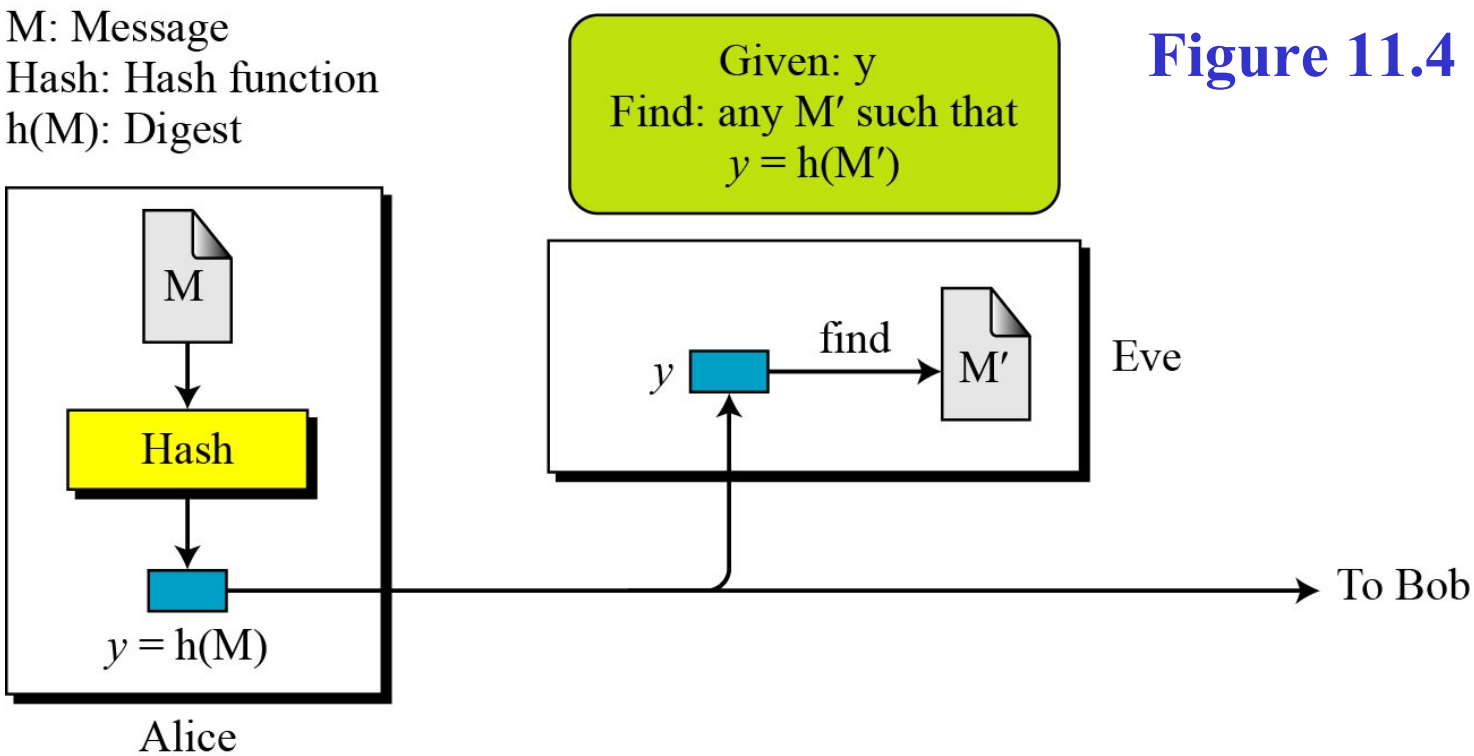


Figure 11.4 ***Preimage***

11.1.5 Continued

Example 11.1

Can we use a conventional lossless compression method such as *StuffIt* as a cryptographic hash function?

Solution

We cannot. A lossless compression method creates a compressed message that is reversible.

Example 11.2

Can we use a checksum function as a cryptographic hash function?

Solution

We cannot. A checksum function is not preimage resistant, Eve may find several messages whose checksum matches the given one.

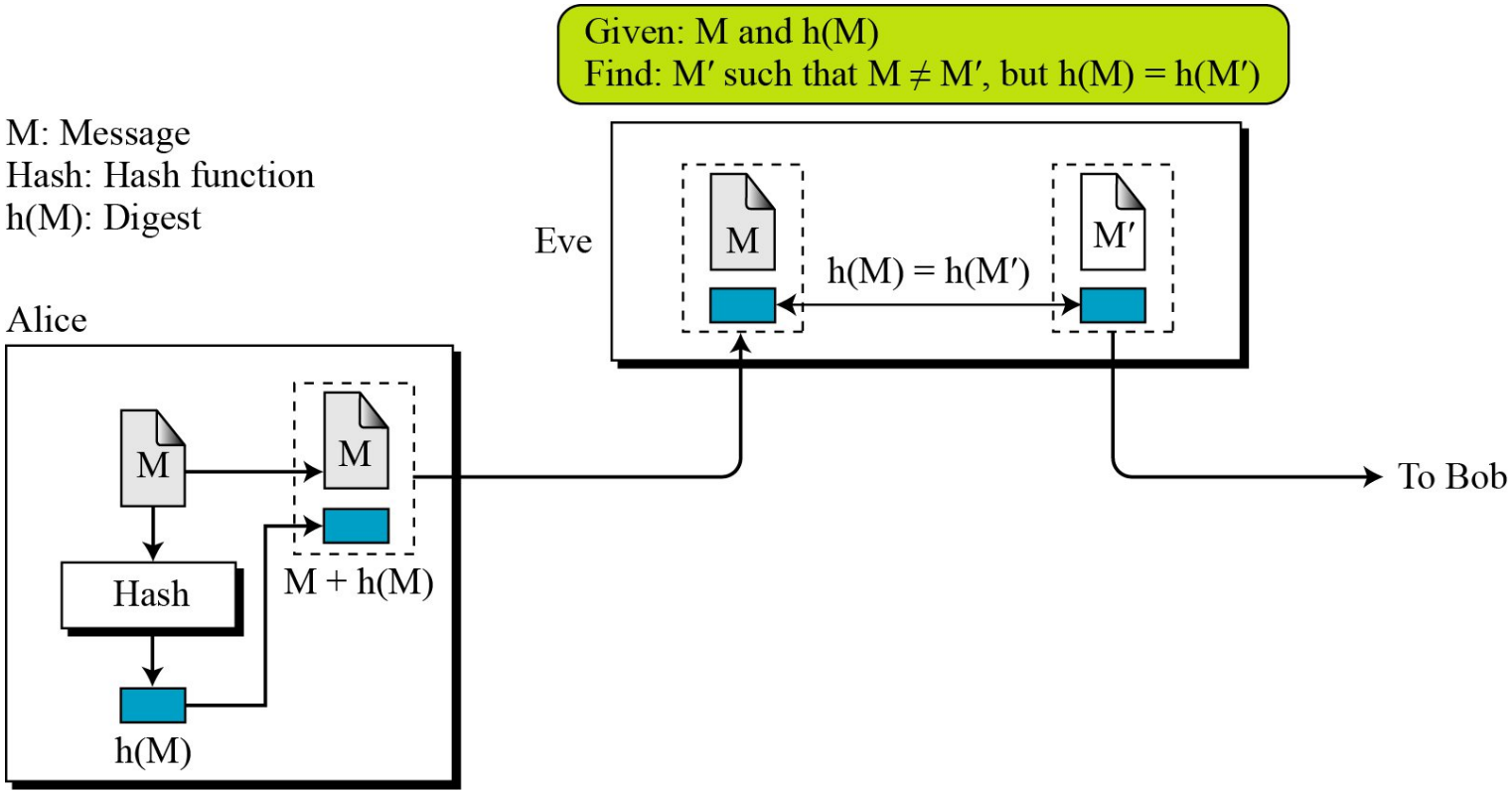
11.1.5 Continued

Second Preimage Resistance

Second Preimage Attack

Given: M and h(M) **Find: $M' \neq M$ such that $h(M) = h(M')$**

Figure 11.5 ***Second preimage***



11.1.5 Continued

Collision Resistance

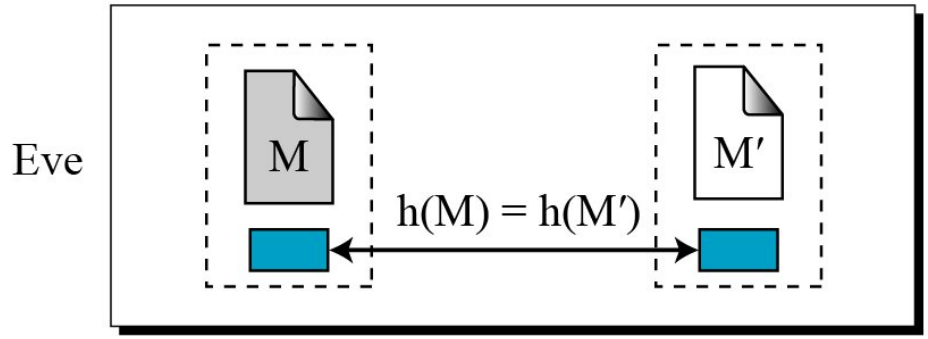
Collision Attack

Given: none **Find: $M' \neq M$ such that $h(M) = h(M')$**

Figure 11.6 ***Collision***

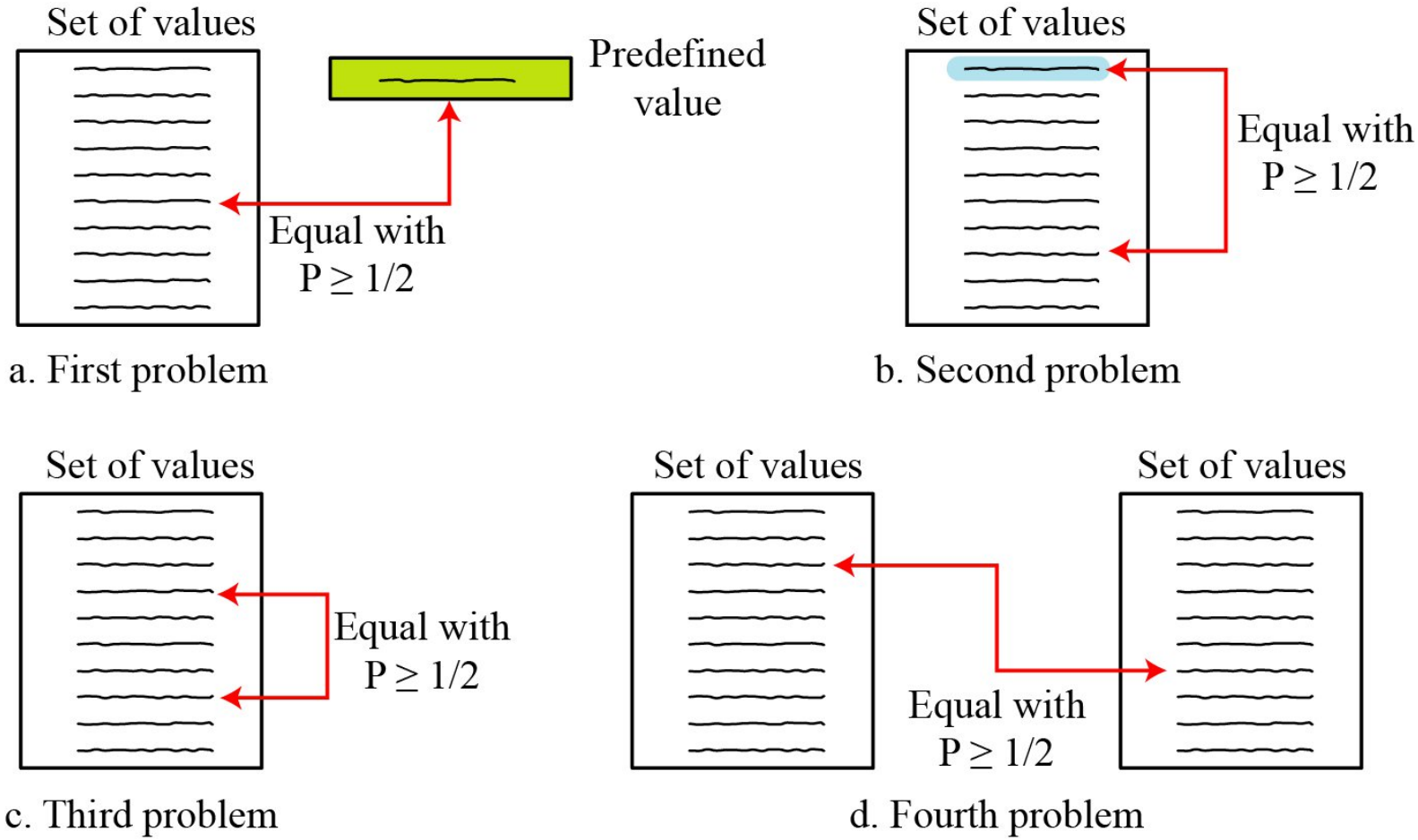
M: Message
Hash: Hash function
h(M): Digest

Find: M and M′ such that $M \neq M'$, but $h(M) = h(M')$



11.2.2 Birthday Problems

Figure 11.7 ***Four birthday problems***



11.2.2 Continued

Summary of Solutions

Solutions to these problems are given in Appendix E for interested readers; The results are summarized in Table 11.3.

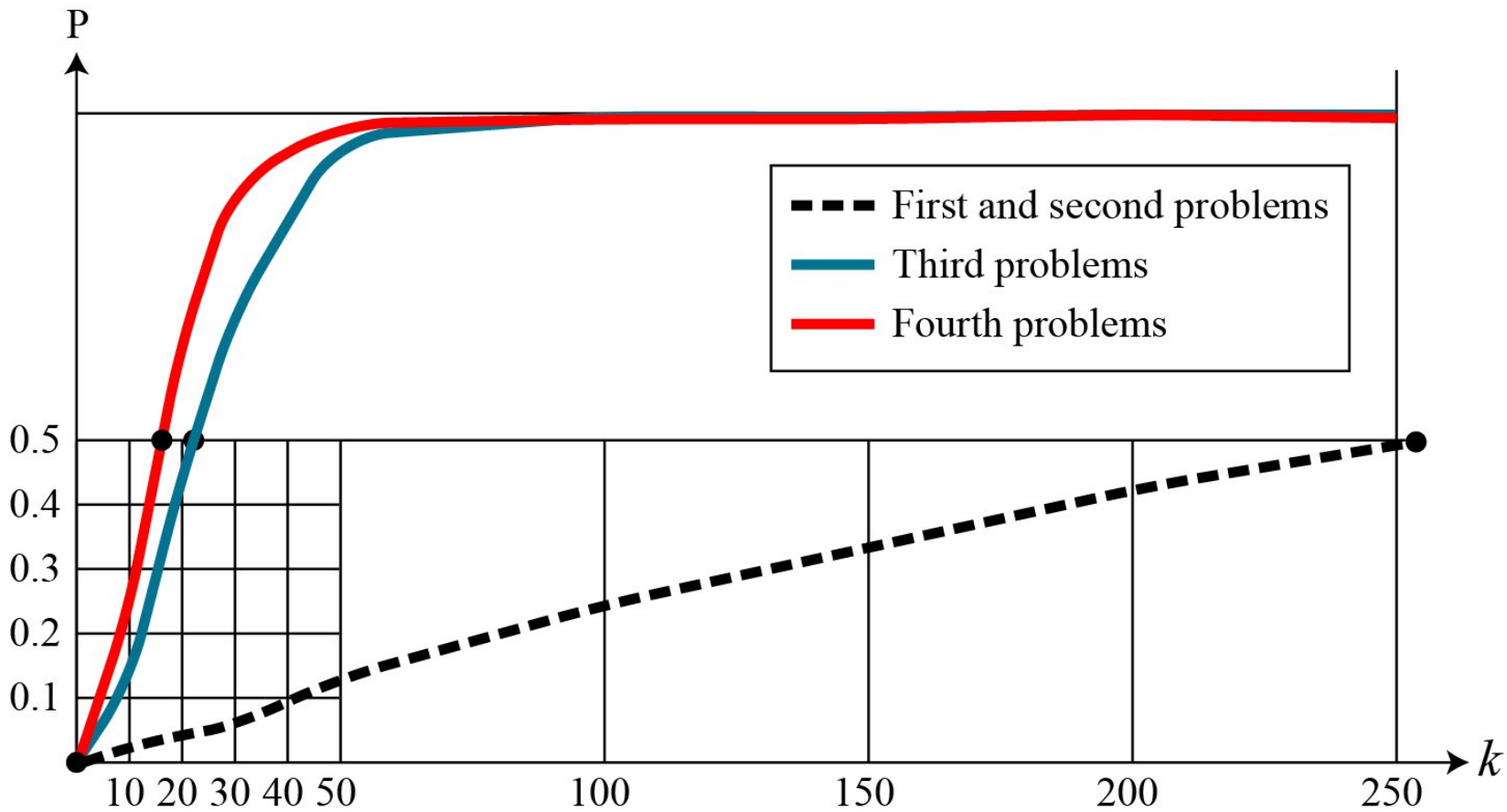
Table 11.3 *Summarized solutions to four birthday problems*

Problem	*Probability*	*General value for k*	*Value of k with P = 1/2*	*Number of students (N = 365)*
1	$P \approx 1 - e^{-k/N}$	$k \approx \ln[1/(1-P)] \times N$	$k \approx 0.69 \times N$	253
2	$P \approx 1 - e^{-(k-1)/N}$	$k \approx \ln[1/(1-P)] \times N + 1$	$k \approx 0.69 \times N + 1$	254
3	$P \approx 1 - e^{k(k-1)/2N}$	$k \approx \{2 \ln [1/(1-P)]\}^{1/2} \times N^{1/2}$	$k \approx 1.18 \times N^{1/2}$	23
4	$P \approx 1 - e^{-k^2/2N}$	$k \approx \{\ln [1/(1-P)]\}^{1/2} \times N^{1/2}$	$k \approx 0.83 \times N^{1/2}$	16

11.2.2 Continued

Comparison

Figure 11.8 *Graph of four birthday problem*



11.2.3 Continued

Second Preimage Attack.

Algorithm 11.2 *Second preimage attack*

```
Second_Preimage_Attack (D, M)
{
   for (i = 1 to k −1)
   {
        create (M [i])
        T ← h (M [i])                    // T is a temporary digest
        if (T = D) return M [i]
   }
   return failure
}
```

The difficulty of a second preimage attack is proportional to 2^n.

Collision Attack

Algorithm 11.3 *Collision attack*

```
Collision_Attack
{
   for (i = 1 to k )
   {
      create (M[i])
      D[i] ← h (M[i])                    // D [i] is a list of created digests
      for (j = 1 to i − 1)
      {
         if (D[i] = D[j]) return (M[i] and M[j])
      }
   }
   return failure
}
```

The difficulty of a collision attack is proportional to $2^{n/2}$.

Alternate Collision Attack

Algorithm 11.4 *Alternate collision attack*

```
Alternate_Collision_Attack (M [k], M′[k])
{
   for (i = 1 to k )
   {
      D[i] ← h (M[i])
      D′[i] ← h (M′[i])
      if (D [i] = D′[j]) return (M[i], M′[j])
   }
   return failure
}
```

The difficulty of an alternative collision attack is proportional to $2^{n/2}$.

11.2.3 Continued

Summary of Attacks

Table 11.4 shows the level of difficulty for each attack if the digest is n bits.

Table 11.4 *Levels of difficulties for each type of attack*

Attack	*Value of k with P=1/2*	*Order*
Preimage	$k \approx 0.69 \times 2^n$	2^n
Second preimage	$k \approx 0.69 \times 2^n + 1$	2^n
Collision	$k \approx 1.18 \times 2^{n/2}$	$2^{n/2}$
Alternate collision	$k \approx 0.83 \times 2^{n/2}$	$2^{n/2}$

12-1 INTRODUCTION

A cryptographic hash function takes a message of arbitrary length and creates a message digest of fixed length. The ultimate goal of this chapter is to discuss the details of the two most promising cryptographic hash algorithms¾ SHA-512 and Whirlpool.

Topics discussed in this section:

12.1.1 Iterated Hash Function

12.1.2 Two Groups of Compression Functions

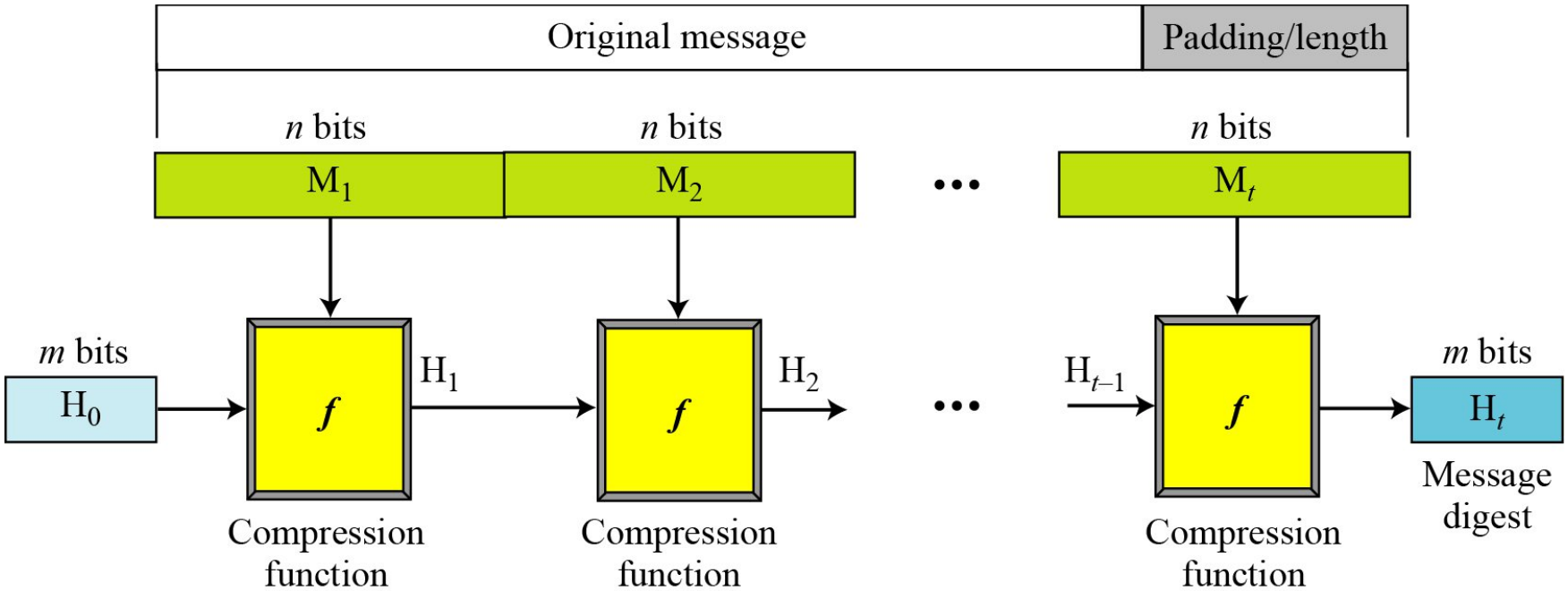
Iterated hash function

- Creating a fixed size digest out of a variable size message
- Creation of such function is achieved through iteration
- Uses fixed size input function [Compression function] , is used necessary number of times
- Compression function compresses n-bit string to a m-bit string where n>m

12.1.1 Iterated Hash Function

Merkle-Damgard Scheme

Figure 12.1 **Merkle-Damgard scheme**



12.1.2 Continued

Table 12.1 *Characteristics of Secure Hash Algorithms (SHAs)*

Characteristics	*SHA-1*	*SHA-224*	*SHA-256*	*SHA-384*	*SHA-512*
Maximum Message size	$2^{64} - 1$	$2^{64} - 1$	$2^{64} - 1$	$2^{128} - 1$	$2^{128} - 1$
Block size	512	512	512	1024	1024
Message digest size	160	224	256	384	512
Number of rounds	80	64	64	80	80
Word size	32	32	32	64	64

12-2 SHA-512

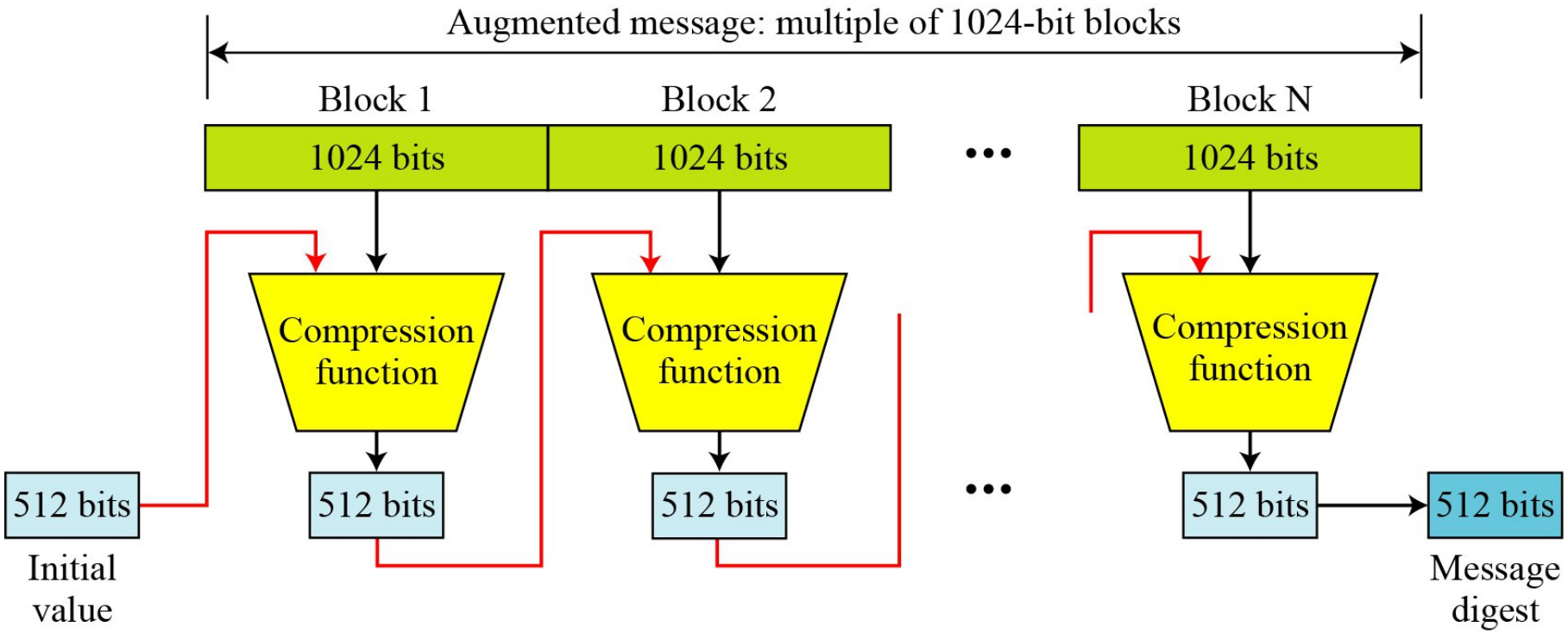
SHA-512 is the version of SHA with a 512-bit message digest. This version, like the others in the SHA family of algorithms, is based on the Merkle-Damgard scheme.

Topics discussed in this section:

12.2.1 Introduction
12.2.2 Compression Function
12.2.3 Analysis

12.2.1 Introduction

Figure 12.6 **Message digest creation SHA-512**



Message Preparation

SHA-512 insists that the length of the original message be less than 2^{128} bits.

Note

SHA-512 creates a 512-bit message digest out of a message less than 2^{128}.

12.2.1 Continued

Example 12.1

This example shows that the message length limitation of SHA-512 is not a serious problem. Suppose we need to send a message that is 2^{128} bits in length. How long does it take for a communications network with a data rate of 2^{64} bits per second to send this message?

Solution

A communications network that can send 2^{64} [2^{64} =18446744073709551616] bits per second is not yet available. Even if it were, it would take many years to send this message. This tells us that we do not need to worry about the SHA-512 message length restriction.

12.2.1 Continued

Example 12.2

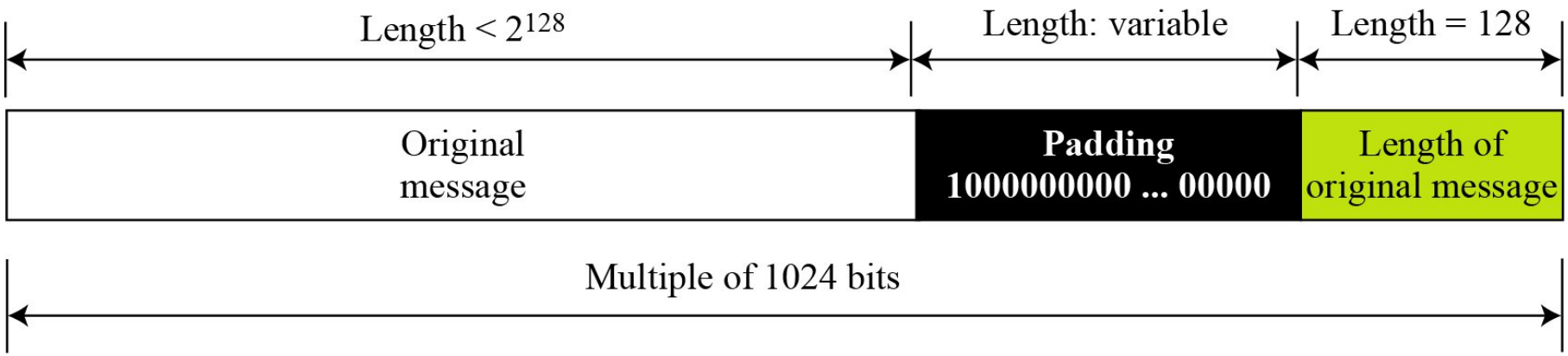
This example also concerns the message length in SHA-512. How many pages are occupied by a message of 2^{128} bits?

Solution

Suppose that a character is 32, or 2^6, bits. Each page is less than 2048, or approximately 2^{12}, characters. So 2^{128} bits need at least $2^{128} / 2^{18}$, or 2^{110}, pages. This again shows that we need not worry about the message length restriction.

12.2.1 Continued

Figure 12.7 **Padding and length field in SHA-512**



12.2.1 Continued

Example 12.3

What is the number of padding bits if the length of the original message is 2590 bits?

Solution

We can calculate the number of padding bits as follows:

$$|P| = (-2590 - 128) \bmod 1024 = -2718 \bmod 1024 = 354$$

The padding consists of one 1 followed by 353 0's.

12.2.1 Continued

Example 12.4

Do we need padding if the length of the original message is already a multiple of 1024 bits?

Solution

Yes we do, because we need to add the length field. So padding is needed to make the new block a multiple of 1024 bits.

12.2.1 Continued

Example 12.5

What is the minimum and maximum number of padding bits that can be added to a message?

Solution

a. The minimum length of padding is 0 and it happens when
$(-M - 128) \bmod 1024$ is 0. This means that $|M| = -128 \bmod 1024 = 896 \bmod 1024$ bits. In other words, the last block in the original message is 896 bits. We add a 128-bit length field to make the block complete.

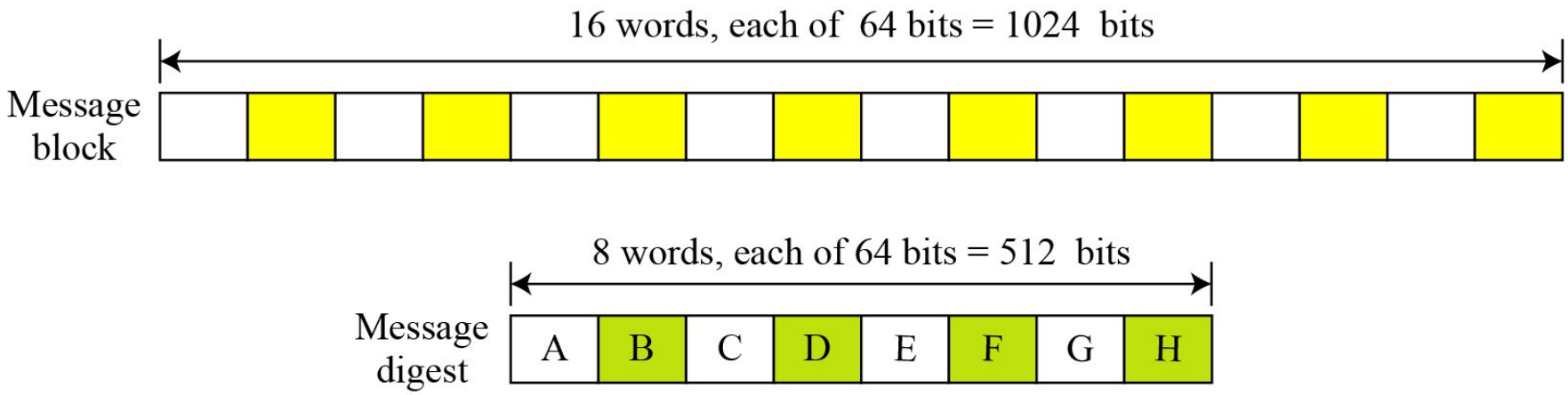
12.2.1 Continued

Example 12.5 Continued

b) The maximum length of padding is 1023 and it happens when $(-|M| - 128) = 1023 \bmod 1024$. This means that the length of the original message is $|M| = (-128 - 1023) \bmod 1024$ or the length is $|M| = 897 \bmod 1024$. In this case, we cannot just add the length field because the length of the last block exceeds one bit more than 1024. So we need to add 897 bits to complete this block and create a second block of 896 bits. Now the length can be added to make this block complete.

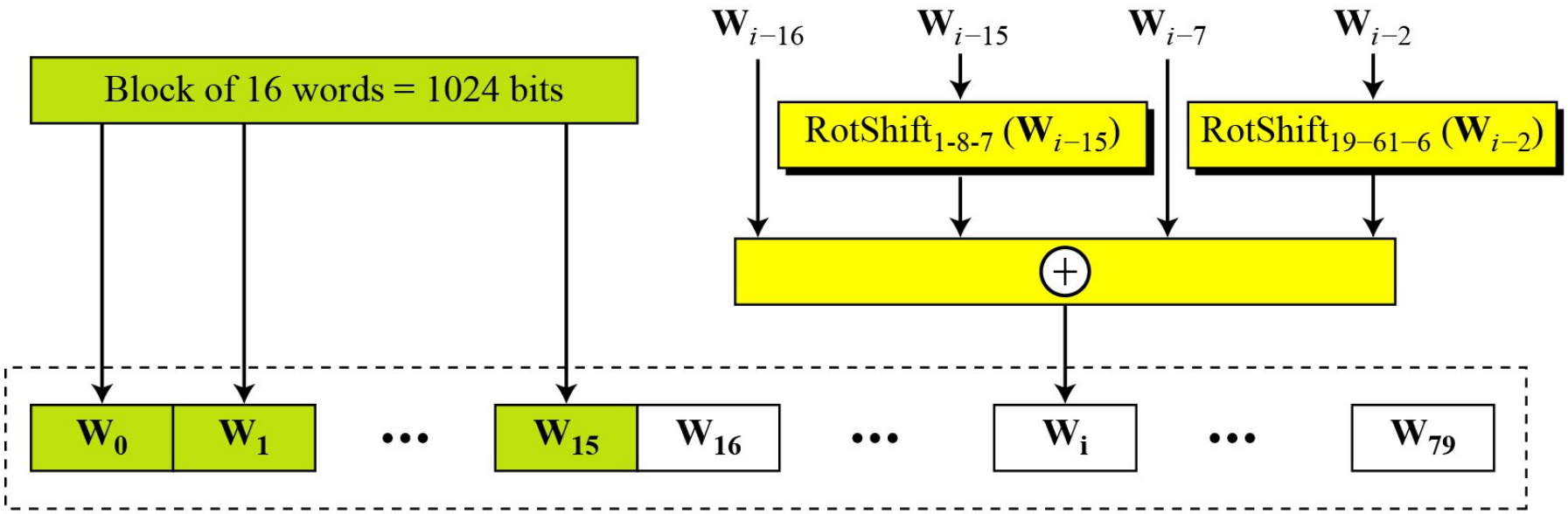
Words

Figure 12.8 A message block and the digest as words



Word Expansion

Figure 12.9 Word expansion in SHA-512



$RotShift_{l-m-n}(x)$: $RotR_l(x) \oplus RotR_m(x) \oplus ShL_n(x)$

$RotR_i(x)$: Right-rotation of the argument x by i bits
$ShL_i(x)$: Shift-left of the argument x by i bits and padding the left by 0's.

12.2.1 Continued

Example 12.6

Show how W60 is made.

Solution

Each word in the range W16 to W79 is made from four previously-made words. W60 is made as

$$W_{60} = W_{44} \oplus \text{RotShift}_{1\text{-}8\text{-}7}(W_{45}) \oplus W_{53} \oplus \text{RotShift}_{19\text{-}61\text{-}6}(W_{58})$$

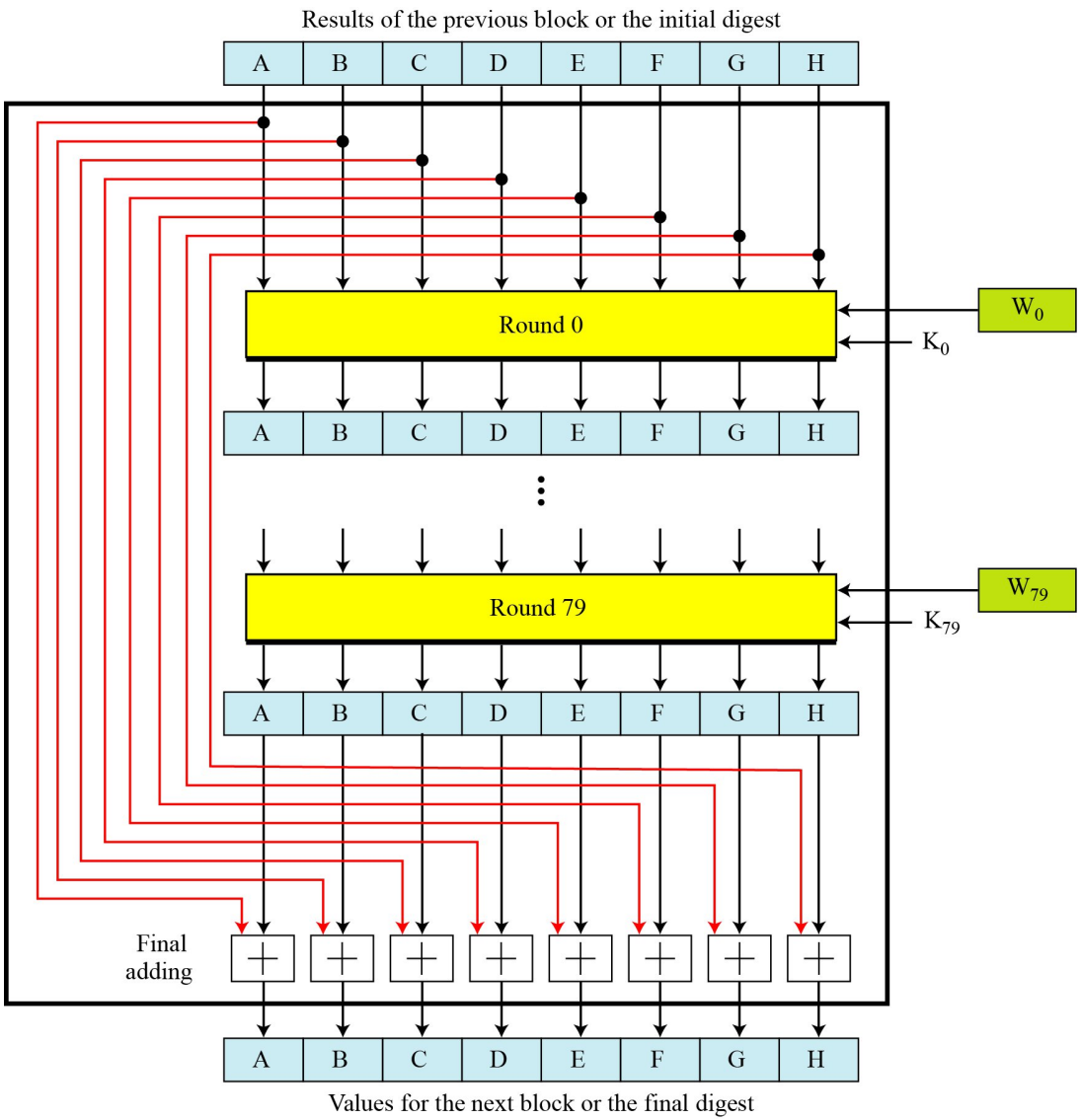
Message Digest Initialization

Table 12.2 *Values of constants in message digest initialization of SHA-512*

Buffer	*Value (in hexadecimal)*	*Buffer*	*Value (in hexadecimal)*
A_0	6A09E667F3BCC908	E_0	510E527FADE682D1
B_0	BB67AE8584CAA73B	F_0	9B05688C2B3E6C1F
C_0	3C6EF372EF94F828	G_0	1F83D9ABFB41BD6B
D_0	A54FE53A5F1D36F1	H_0	5BE0CD19137E2179

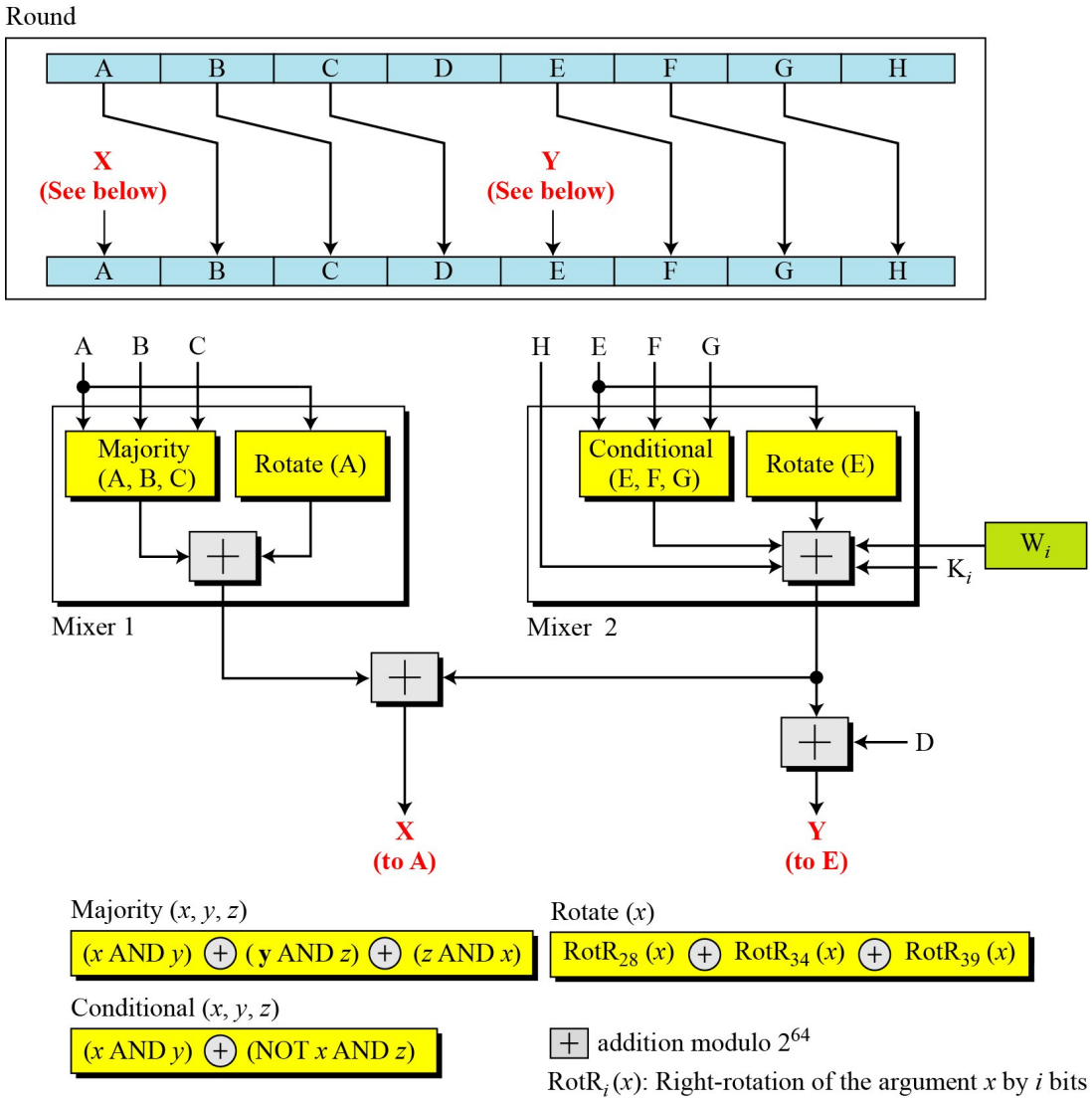
12.2.2 Compression Function

Figure 12.10 Compression function in SHA-512



12.2.2 Continued

Figure 12.11 Structure of each round in SHA-512



12.2.2 Continued

Majority

$$(A_j \text{ AND } B_j) \oplus (B_j \text{ AND } C_j) \oplus (C_j \text{ AND } A_j)$$

Conditional

$$(E_j \text{ AND } F_j) \oplus (\text{NOT } E_j \text{ AND } G_j)$$

Rotate

$$\text{Rotate (A): } \text{RotR}_{28}(A) \oplus \text{RotR}_{34}(A) \oplus \text{RotR}_{29}(A)$$

$$\text{Rotate (E): } \text{RotR}_{28}(E) \oplus \text{RotR}_{34}(E) \oplus \text{RotR}_{29}(E)$$

12.2.2 Continued

Table 12.3 *Eighty constants used for eighty rounds in SHA-512*

428A2F98D728AE22	7137449123EF65CD	B5C0FBCFEC4D3B2F	E9B5DBA58189DBBC
3956C25BF348B538	59F111F1B605D019	923F82A4AF194F9B	AB1C5ED5DA6D8118
D807AA98A3030242	12835B0145706FBE	243185BE4EE4B28C	550C7DC3D5FFB4E2
72BE5D74F27B896F	80DEB1FE3B1696B1	9BDC06A725C71235	C19BF174CF692694
E49B69C19EF14AD2	EFBE4786384F25E3	0FC19DC68B8CD5B5	240CA1CC77AC9C65
2DE92C6F592B0275	4A7484AA6EA6E483	5CB0A9DCBD41FBD4	76F988DA831153B5
983E5152EE66DFAB	A831C66D2DB43210	B00327C898FB213F	BF597FC7BEEF0EE4
C6E00BF33DA88FC2	D5A79147930AA725	06CA6351E003826F	142929670A0E6E70
27B70A8546D22FFC	2E1B21385C26C926	4D2C6DFC5AC42AED	53380D139D95B3DF
650A73548BAF63DE	766A0ABB3C77B2A8	81C2C92E47EDAEE6	92722C851482353B
A2BFE8A14CF10364	A81A664BBC423001	C24B8B70D0F89791	C76C51A30654BE30
D192E819D6EF5218	D69906245565A910	F40E35855771202A	106AA07032BBD1B8
19A4C116B8D2D0C8	1E376C085141AB53	2748774CDF8EEB99	34B0BCB5E19B48A8
391C0CB3C5C95A63	4ED8AA4AE3418ACB	5B9CCA4F7763E373	682E6FF3D6B2B8A3
748F82EE5DEFB2FC	78A5636F43172F60	84C87814A1F0AB72	8CC702081A6439EC
90BEFFFA23631E28	A4506CEBDE82BDE9	BEF9A3F7B2C67915	C67178F2E372532B
CA273ECEEA26619C	D186B8C721C0C207	EADA7DD6CDE0EB1E	F57D4F7FEE6ED178
06F067AA72176FBA	0A637DC5A2C898A6	113F9804BEF90DAE	1B710B35131C471B
28DB77F523047D84	32CAAB7B40C72493	3C9EBE0A15C9BEBC	431D67C49C100D4C
4CC5D4BECB3E42B6	4597F299CFC657E2	5FCB6FAB3AD6FAEC	6C44198C4A475817

12.2.2 Continued

There are 80 constants, K_0 to K_{79}, each of 64 bits. Similar These values are calculated from the first 80 prime numbers (2, 3,…, 409). For example, the 80th prime is 409, with the cubic root $(409)^{1/3}$ = 7.42291412044. Converting this number to binary with only 64

$$(111.0110\ 1100\ 0100\ 0100\ \ldots\ 0111)_2 \rightarrow (7.6C44198C4A475817)_{16}$$

The fraction part: $(6C44198C4A475817)_{16}$

12.2.2 Continued

Example 12.7

We apply the Majority function on buffers A, B, and C. If the leftmost hexadecimal digits of these buffers are 0x7, 0xA, and 0xE, respectively, what is the leftmost digit of the result?

Solution

The digits in binary are 0111, 1010, and 1110.

a. The first bits are 0, 1, and 1. The majority is 1.

b. The second bits are 1, 0, and 1. The majority is 1.

c. The third bits are 1, 1, and 1. The majority is 1.

d. The fourth bits are 1, 0, and 0. The majority is 0.

The result is 1110, or 0xE in hexadecimal.

12.2.2 Continued

Example 12.8

We apply the Conditional function on E, F, and G buffers. If the leftmost hexadecimal digits of these buffers are 0x9, 0xA, and 0xF respectively, what is the leftmost digit of the result?

Solution

The digits in binary are 1001, 1010, and 1111.

a. The first bits are 1, 1, and 1. The result is F_1, which is 1.
b. The second bits are 0, 0, and 1. The result is G_2, which is 1.
c. The third bits are 0, 1, and 1. The result is G_3, which is 1.

d. The fourth bits are 1, 0, and 1. The result is F_4, which is 0.

The result is 1110, or 0xE in hexadecimal.

12.2.3 Analysis

With a message digest of 512 bits, SHA-512 expected to be resistant to all attacks, including collision attacks.